

UNIVERSITY OF MANAGEMENT AND TECHNOLOGY, LAHORE

School of Systems and Technology

Department of Computer Science

CS406 - DIGITAL IMAGE PROCESSING
HOMEWORK ASSIGNMENT # 1 CLO-1

Instructor	Dr. Jameel Ahmad, Associate Professor
Department	Computer Science, School of Systems and Technology
Contact	Jameel.ahmad@umt.edu.pk
Topics Covered	Image Reading & Writing, Image Arithmetic, Image Sampling & Quantization, Generation of a 2-D Gaussian Smoothing Function
Submission Deadline	August 10, 2026, 11:59 PM (midnight)
Total Marks	100 (20 marks per question)
Tools Required	Python 3.x, OpenCV (opencv-python), NumPy, Matplotlib

“Vision is the art of seeing what is invisible to others.”

Contents

Instructions to Students	1
1 Question 1 — Image Reading and Writing (20 marks)	2
1.1 1(a) Numerical (8 marks)	2
1.2 1(b) Programming (12 marks)	2
2 Question 2 — Image Arithmetic (20 marks)	2
2.1 2(a) Numerical (8 marks)	2
2.2 2(b) Programming (12 marks)	3
3 Question 3 — Image Sampling (20 marks)	3
3.1 3(a) Numerical (8 marks)	3
3.2 3(b) Programming (12 marks)	3
4 Question 4 — Image Quantization (20 marks)	4
4.1 4(a) Numerical (8 marks)	4
4.2 4(b) Programming (12 marks)	4
5 Question 5 — Generation of a 2-D Gaussian Smoothing Function (20 marks)	5
5.1 5(a) Numerical (8 marks)	5
5.2 5(b) Programming (12 marks)	5
Marking Rubric	5
A Appendix: Required Python Libraries	6
B Appendix: Sample Directory Structure	6
Conclusion	6

Instructions to Students

1. This assignment contains 5 questions. Each question has two parts: a numerical (hand-worked) part and a programming part based on Python and OpenCV.
2. Attempt the numerical parts by hand first (show all steps/formulas), then verify your answer with a Python script.
3. Submit a single ZIP file named StudentID_HW1.zip containing:
 - A PDF report (StudentID_HW1_Report.pdf) with your numerical solutions, code screenshots/output, and discussion for every question.
 - Separate, runnable .py files for each question, named Q1.py, Q2.py, Q3.py, Q4.py, Q5.py.
 - Any images you generated or used, placed in a folder named images/.
4. Add comments in your code explaining each step.
5. You may use any grayscale image of your choice for the programming parts unless a specific image/-matrix is given in the question. State clearly which image you used.
6. Late submission policy: as per course policy communicated in class; no submissions will be accepted after the deadline unless prior approval is granted by the instructor.

7. Academic integrity: Copied code or reports (from classmates or the internet) will be marked zero for all parties involved, and reported to the Discipline Committee.
8. Use `import cv2`, `import numpy as np`, and `import matplotlib.pyplot as plt` consistently; do not use `cv2.imshow()` if you are working in a notebook — use `matplotlib` for displaying images in your report instead.

1 Question 1 — Image Reading and Writing (20 marks)

1.1 1(a) Numerical (8 marks)

A grayscale image is represented by the following 4×4 intensity matrix (8 bits/pixel, i.e., 256 gray levels):

$$\mathbf{I} = \begin{bmatrix} 12 & 45 & 200 & 255 \\ 33 & 90 & 128 & 180 \\ 5 & 64 & 100 & 210 \\ 220 & 150 & 75 & 20 \end{bmatrix}$$

- i. Calculate the total memory required (in bits and bytes) to store this image assuming 8 bits per pixel (bpp).
- ii. If the same image is instead stored using only 3 bits per pixel, calculate the new storage size in bits and bytes, and compute the compression ratio (original size / compressed size).
- iii. If this 4×4 image is to be saved in an uncompressed BMP file, the file also carries a fixed 54-byte header. Calculate the total file size in bytes for part (i).
- iv. Briefly explain, in 2–3 sentences, why `cv2.imread()` by default reads even a single-channel grayscale image as a 3-channel BGR array, and what flag prevents this.

1.2 1(b) Programming (12 marks)

Write a Python/OpenCV program that:

- i. Reads any color image from disk using `cv2.imread()`.
- ii. Prints its shape, data type (`dtype`), and total number of pixels.
- iii. Converts it to grayscale using `cv2.cvtColor()` and displays the original and grayscale images side by side using `matplotlib`.
- iv. Writes the grayscale image to disk in three formats: `.png`, `.jpg` (quality = 50), and `.bmp`, using `cv2.imwrite()`.
- v. Reads back the file sizes of all three saved files (use Python's `os.path.getsize()`) and prints them in a table.
- vi. Comments on which format is smallest/largest and why (lossy vs. lossless compression).

2 Question 2 — Image Arithmetic (20 marks)

2.1 2(a) Numerical (8 marks)

Consider two 3×3 image patches (8-bit, values 0–255):

$$\mathbf{A} = \begin{bmatrix} 250 & 10 & 90 \\ 60 & 200 & 30 \\ 15 & 5 & 180 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 20 & 250 & 40 \\ 80 & 90 & 220 \\ 10 & 240 & 90 \end{bmatrix}$$

- i. Compute $A + B$ pixel-by-pixel using modulo-256 (wrap-around) arithmetic, i.e., $\text{result} = (a + b) \bmod 256$. Show the full 3×3 result and clearly mark every pixel where wrap-around occurred.
- ii. Compute $A + B$ using saturation arithmetic (clip to the range $[0, 255]$), i.e., $\text{result} = \min(a + b, 255)$. Show the full 3×3 result.
- iii. Compute $A - B$ using saturation arithmetic (clip to $[0, 255]$), i.e., $\text{result} = \max(a - b, 0)$.
- iv. In 2–3 sentences, explain why saturation arithmetic is preferred over wrap-around arithmetic for real image processing applications.

2.2 2(b) Programming (12 marks)

Write a Python/OpenCV program that:

- i. Creates two 3×3 or 5×5 NumPy arrays with `dtype=np.uint8` using the values from part (a) (or reads two real grayscale images of the same size).
- ii. Computes their sum using plain NumPy addition ($A + B$) and prints the result — this demonstrates wrap-around behavior.
- iii. Computes their sum using `cv2.add(A, B)` and prints the result — this demonstrates saturation behavior.
- iv. Computes the difference using `cv2.subtract(A, B)` and $A - B$ (NumPy) and compares both outputs.
- v. For two real grayscale images of the same size, also demonstrate `cv2.addWeighted()` to blend them with weights 0.7 and 0.3, and display the blended result.
- vi. Discuss (in code comments or printed text) where the NumPy result and OpenCV result differ, and why.

3 Question 3 — Image Sampling (20 marks)

3.1 3(a) Numerical (8 marks)

A single row of an image has the following 8 intensity values (this represents an 8-point discretely sampled signal):

$$f(n) = [10, 100, 250, 40, 30, 220, 200, 15], \quad n = 0, 1, \dots, 7$$

- i. Down-sample this row by a factor of 2 (keep every 2nd sample starting at $n = 0$). Write out the resulting 4-point sequence.
- ii. Down-sample the same original row by a factor of 4 (keep every 4th sample). Write out the resulting 2-point sequence.
- iii. Using nearest-neighbor interpolation, up-sample the factor-2 down-sampled sequence from part (i) back to 8 points. Compare it, point-by-point, to the original sequence and compute the Mean Squared Error (MSE).
- iv. In 2–3 sentences, explain the concept of aliasing and why down-sampling without a low-pass (anti-aliasing) filter can produce it, referring to your numerical example.

3.2 3(b) Programming (12 marks)

Write a Python/OpenCV program that:

- i. Reads a grayscale image of size at least 256×256 .

- ii. Down-samples it by factors of 2, 4, and 8 by direct pixel skipping (array slicing, e.g., `img[::2, ::2]`), with no filtering beforehand.
- iii. Repeats the down-sampling at the same factors, but this time applies a `cv2.GaussianBlur()` before sub-sampling (anti-aliasing).
- iv. Displays all six results (3 unfiltered + 3 filtered) in a single matplotlib figure with titles.
- v. For each down-sampled/re-upscaled version (use `cv2.resize()` with `INTER_NEAREST` to restore original size), computes the MSE and PSNR with respect to the original image, and tabulates the results for all 6 cases.
- vi. Comments on which method (filtered vs. unfiltered) gives better PSNR and why.

4 Question 4 — Image Quantization (20 marks)

4.1 4(a) Numerical (8 marks)

Given the following 8-bit pixel values (0–255):

$$r = [18, 63, 97, 130, 175, 210, 240, 5]$$

Uniform quantization to L gray levels uses step size $\Delta = 256/L$, and the quantized value for pixel r is:

$$q(r) = \lfloor r/\Delta \rfloor \cdot \Delta + \Delta/2 \quad (\text{rounded to nearest integer})$$

- i. Quantize each value of r to $L = 4$ levels (2 bits/pixel). Show Δ , the computation of $\lfloor r/\Delta \rfloor$ for each pixel, and the final quantized value $q(r)$.
- ii. Quantize each value of r to $L = 2$ levels (1 bit/pixel, i.e., binary thresholding at mid-gray). Show the final quantized values.
- iii. For the $L = 4$ case, compute the quantization error $e(r) = r - q(r)$ for every pixel, and the Mean Squared Quantization Error (MSQE) over all 8 pixels.
- iv. In 2–3 sentences, explain the general relationship between number of quantization levels L and MSQE, and what visual artifact appears in images quantized to very few levels.

4.2 4(b) Programming (12 marks)

Write a Python/OpenCV program that:

- i. Defines a function `quantize(img, L)` that quantizes an 8-bit grayscale image to L gray levels using the formula in part (a) (fully vectorized with NumPy — no pixel-wise Python loops).
- ii. Reads a grayscale image and produces quantized versions for $L = 2, 4, 8, 16, 32, 64, 128, 256$.
- iii. Displays all 8 versions in a single matplotlib grid with the value of L as the title of each subplot ("false contouring" should be visible for small L).
- iv. Computes the MSQE between the original image and each quantized version, and plots MSQE vs. L using matplotlib.
- v. Comments on the trend of the plot and relates it to your numerical result in part (a).

5 Question 5 — Generation of a 2-D Gaussian Smoothing Function (20 marks)

5.1 5(a) Numerical (8 marks)

The 2-D Gaussian function (zero mean) is defined as:

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

- For $\sigma = 1$ and a 5×5 window where $x, y \in \{-2, -1, 0, 1, 2\}$, compute the un-normalized value of $G(x, y)$ for every one of the 25 (x, y) positions. Present your answer as a 5×5 matrix (4 decimal places).
- Normalize the kernel so that all 25 values sum to exactly 1 (divide every entry by the sum of all un-normalized entries). Present the final normalized 5×5 kernel (4 decimal places).
- Verify by hand (show the sum) that your normalized kernel indeed sums to 1.000 ($\pm$ rounding error).
- In 2–3 sentences, explain why the Gaussian kernel must be normalized before it is used as a smoothing/convolution filter on an image.

5.2 5(b) Programming (12 marks)

Write a Python/OpenCV program that:

- Implements a function `gaussian_kernel_2d(size, sigma)` from scratch using only NumPy (no `cv2.getGaussianKernel`) that returns a normalized $\text{size} \times \text{size}$ 2-D Gaussian kernel, following the formula above.
- Prints the 5×5 kernel for $\sigma = 1$ and confirms it matches your hand-computed answer from part 5(a)(ii).
- Independently generates a 5×5 Gaussian kernel using OpenCV's built-in `cv2.getGaussianKernel(5, 1)` (a 1-D kernel) and forms the 2-D kernel via the outer product of the 1-D kernel with itself. Compares this result numerically (e.g., using `np.allclose`) with your from-scratch kernel in (i).
- Applies your from-scratch kernel to a grayscale image using `cv2.filter2D()`, and separately blurs the same image using `cv2.GaussianBlur(img, (5,5), 1)`.
- Displays the original image, your `filter2D` result, and the `GaussianBlur` result side by side, and computes the MSE between your result and OpenCV's built-in result (it should be very close to 0).
- Repeats the kernel generation and visualization (as a 3-D surface plot using `matplotlib`) for $\sigma = 0.5, 1, 2$, and 5, and briefly discusses (2–3 sentences) how increasing σ affects the shape of the kernel and the amount of blurring.

Marking Rubric (per question, 20 marks)

Component	Marks
Numerical part — correct method/formula shown	4
Numerical part — correct final values	4
Programming part — correct, runnable code	6
Programming part — correct output/plots	4
Discussion/explanation quality	2
Total	100

A Appendix: Required Python Libraries

To run the programs in this assignment, you need to install the following Python packages:

```
1 # Install required packages
2 pip install opencv-python
3 pip install numpy
4 pip install matplotlib
```

Listing 1: Installation Commands

B Appendix: Sample Directory Structure

The submission should follow this structure:

```
StudentID_HW1.zip
|-- StudentID_HW1_Report.pdf
|-- Q1.py
|-- Q2.py
|-- Q3.py
|-- Q4.py
|-- Q5.py
'-- images/
    |-- image.jpg
    |-- image1.jpg
    '-- image2.jpg
```

Conclusion

This homework assignment covers fundamental concepts in digital image processing including:

- Image I/O operations and file formats
- Image arithmetic operations and their implications
- Sampling and aliasing effects
- Quantization and false contouring
- Gaussian filtering and kernel generation

All solutions should be implemented using Python with OpenCV, NumPy, and Matplotlib libraries. Ensure that code is properly commented and results are clearly documented in the report.

End of Homework Assignment 1 — CS406 Digital Image Processing

Dr. Jameel Ahmad